SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: Theory of Computation **COURSE CODE: CSA1304**

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 5 **Total Marks:25**
Duration : 1 Hour

AT1- Application Based Question

Code of Conduct:

I GIFTSON KEVIN MATTHEW.Y (192511167) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIFTSON KEVIN MATTHEW.Y

SIMATS ENGINEERING
ASSESSMENT TOOL

1. In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable. Design a **Deterministic Finite Automaton (DFA)** over the alphabet Σ = {0,1} to validate passwords that **end with the pattern "01"**.Further, **minimize the DFA** and analyze how the optimized design improves system efficiency in real-world hardware implementations

2. In web applications, validating user input is essential to ensure correctness, security, and performance. Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet Σ = {a, b}.Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications**.**

3. In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role. Demonstrate how the regular expression `a*b*` can be converted into an equivalent finite automaton (FA).Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

4. In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language $L= \{ a^n b^n \mid n \geq 0 \}$ cannot be recognized by any finite automaton. Further, analyze the implications of this limitation in the design of modern compilers.

5. Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules. Explain how the **closure properties of regular languages** enable the combination of multiple detection rules. Further, justify why the resulting language after applying these operations **remains regular**, ensuring efficient implementation using finite automata

SIMATS ENGINEERING
ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING
ASSESSMENT TOOL

Question 1. Design and Minimize a DFA for Passwords Ending with "01"

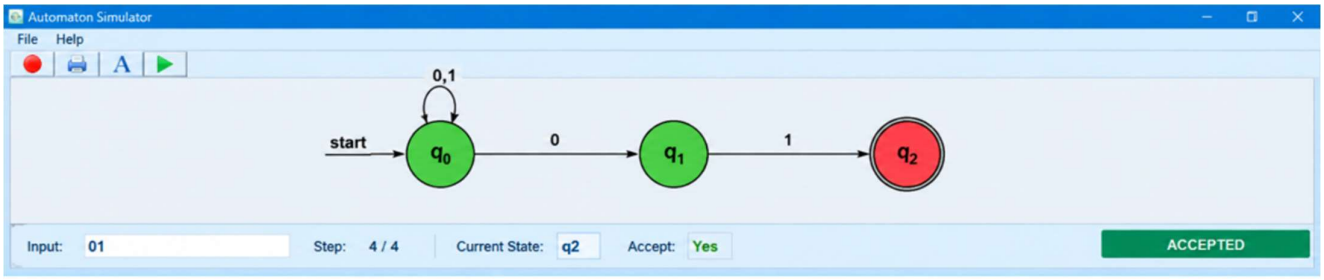
Answer:

A Deterministic Finite Automaton (DFA) is a finite-state machine that reads an input string one symbol at a time and determines whether the string is accepted or rejected. Here, the alphabet is \(\Sigma=\{0,1\}\), and the DFA must accept all binary strings that end with 01.

1. Construction of the DFA

We need three states:

- (q0): Start state; the required pattern has not been matched.
- (q1): The last symbol read is 0.
- (q2): The string ends with 01. This is the final state.



3. Transition table

Present state	Input 0	Input 1
→ (q0)	(q1)	(q0)
(q1)	(q1)	(q2)
*(q2)	(q1)	(q0)

Here, (q0) is the initial state and (q2) is the accepting state.

4. Minimization of the DFA

The DFA is already minimized because all three states represent different situations.

- (q0) and (q1) are different because input 1 leads to acceptance from (q1), but not from (q0).
- (q1) and (q2) are different because (q2) is accepting, whereas (q1) is not.
- (q0) and (q2) are different because one is non-accepting and the other is accepting.

Therefore, no two states can be merged. The minimum DFA contains 3 states.

5. Example

Input string	Result	Reason
101	Accepted	Ends with 01
1101	Accepted	Ends with 01
100	Rejected	Ends with 00
111	Rejected	Does not end with 01

6. Real-world application

In embedded security devices and authentication systems, a minimized DFA requires fewer states and less memory. It can process input bits using simple state transitions, reducing hardware complexity and potentially lowering power consumption. However, this DFA checks only the ending pattern; it does not provide complete password security.

Conclusion: The minimized DFA with three states efficiently recognizes binary strings ending in 01.

Question 2. Regular Expression for the Pattern “aaaabb”

Answer:

A regular expression is a way of representing a pattern of characters. In web applications, it is useful for checking whether user input follows a specified format.

1. Given information

- Alphabet: (Sigma={a,b})
- Required pattern: aaaabb

The input must contain exactly four a characters followed by two b characters.

2. Regular expression

The required regular expression is:

[boxed{texttt{^aaaabb$}}]

Here:

- ^ represents the beginning of the input.
- aaaa represents exactly four occurrences of a.
- bb represents exactly two occurrences of b.
- $ represents the end of the input.

The beginning and ending anchors ensure that no extra characters appear before or after the required pattern.

3. Examples of validation

Input	Result	Explanation
aaaabb	Valid	Exact match
aaaaab	Invalid	Five a characters
Aaabb	Invalid	Only three a characters
aaaabbb	Invalid	Three b characters
bbaaaa	Invalid	Incorrect order

4. Role of regular languages in web applications

Regular expressions and regular languages are useful for validating data before it is processed further.

1. Input validation: They check whether an input follows the required format.
2. Data correctness: They help identify missing, extra, or incorrectly ordered characters.
3. Security: They can reject inputs that do not match an expected format, although regex alone cannot prevent every security attack.
4. Performance: Regular expressions can be implemented using finite automata, allowing efficient pattern matching for suitable regular expressions.

For example, a web application may use regular expressions to validate product codes, identification numbers, or fixed-format user inputs. The pattern should be designed according to the actual validation requirement.

Conclusion: The regular expression ^aaaabb$ accepts only the exact string aaaabb and rejects all other inputs.

Question 3. Convert the Regular Expression (a^*b^*) into a Finite Automaton

Answer:

A regular expression describes a set of strings, while a finite automaton recognizes strings belonging to that set. The regular expression (a^*b^*) represents zero or more a characters followed by zero or more b characters.

1. Understanding the expression

- (a^*): Zero or more occurrences of a.
- (b^*): Zero or more occurrences of b.

Therefore, valid strings include:

[epsilon, a, aa, b, bb, ab, aab, aaabb]

The symbol (epsilon) represents the empty string.

A string such as ba is invalid because an a cannot occur after a b in this pattern.

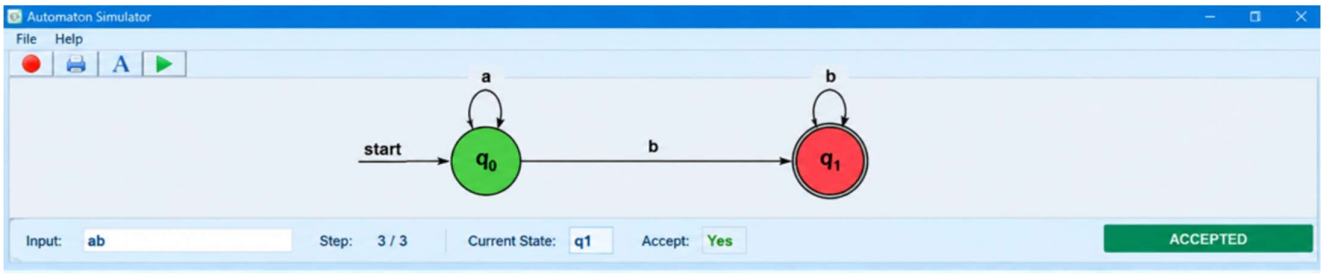
2. Construction of the finite automaton

We construct a DFA with three states:

- (q0): Initial state; reads zero or more a characters. It is also accepting.
- (q1): Reads zero or more b characters after the a portion. It is accepting.
- (qd): Dead state, entered when an a occurs after a b.

SIMATS ENGINEERING
ASSESSMENT TOOL

3. State transition diagram



4. Transition table

Present state	Input (a)	Input (b)
→ *(q0)	(q0)	(q1)
*(q1)	(qd)	(q1)
(qd)	(qd)	(qd)

The accepting states are (q0) and (q1).

5. Working of the automaton

Consider the input aabb.

1. Start at (q0).
2. Read the first a; remain in (q0).
3. Read the second a; remain in (q0).
4. Read the first b; move to (q1).
5. Read the second b; remain in (q1).
6. The input ends in an accepting state, so the string is accepted.

For the input aba, the automaton moves from (q0) to (q1) on b, then to (qd) on a. Therefore, the string is rejected.

6. Why regular expressions and finite automata are equivalent

Regular expressions and finite automata are equivalent because both describe and recognize exactly the class of regular languages. A regular expression can be converted into an equivalent NFA or DFA, and a finite automaton can be represented by a regular expression.

In real-time systems, regular expressions can describe suspicious patterns, while finite automata process input symbols and identify matches. This is useful in intrusion detection, compiler lexical analysis, and monitoring tools.

Conclusion: The DFA with two accepting states and one dead state recognizes the language (a^*b^*).

SIMATS ENGINEERING
ASSESSMENT TOOL

Question 4. Use the Pumping Lemma to Prove That \(L=\{a^n b^n\mid n\geq0\}\) Is Not Regular

Answer:

The pumping lemma is a property of regular languages that can be used to prove that certain languages are not regular.

The given language is:

[L={a^n b^n mid n geq0}]

It contains strings with an equal number of a characters followed by b characters, such as (epsilon), ab, aabb, and aaabbb.

1. Assume that the language is regular

Suppose (L) is regular. According to the pumping lemma, there exists a positive integer (p), called the pumping length, such that any string (s) in (L) with length at least (p) can be divided into three parts:

[s=xyz]

The following conditions must hold:

1. (|xy| leq p)
2. (|y|>0)
3. (xy^iz in L) for every integer (i geq0)

2. Select a suitable string

Choose the string:

[s=a^p b^p]

This string belongs to (L) because it contains exactly (p) occurrences of a, followed by exactly (p) occurrences of b.

3. Divide the string into (x), (y), and (z)

Since (|xy|leq p), both (x) and (y) occur within the first (p) characters. These characters contain only a symbols.

Therefore, (y) must contain one or more a symbols. Let (y=a^k), where (kgeq1).

4. Pump the string down

According to the pumping lemma, we can choose (i=0). This removes (y) from the string:

[xy^0z=xz]

The resulting string contains fewer than (p) occurrences of a, but still contains (p) occurrences of b.

Thus, the numbers of a and b are no longer equal.

[xz notin L]

This contradicts the pumping lemma's requirement that the pumped string must belong to (L).

5. Conclusion of the proof

Our assumption that (L) is regular leads to a contradiction. Therefore,

[boxed{L={a^n b^n mid n geq0}text{ is not regular}}]

6. Implications for compiler design

This limitation is important because finite automata have only a fixed number of states and cannot remember an arbitrarily large count of a symbols to compare it with the number of b symbols.

In compiler design:

- Finite automata are useful for lexical analysis, such as recognizing identifiers, keywords, and numeric literals.
- Context-free grammars and pushdown automata can handle nested structures and certain counting relationships that finite automata cannot recognize.
- Syntax analysis uses more powerful mechanisms to check structures such as balanced parentheses and nested expressions.

Conclusion: The pumping lemma proves that (L) is not regular, demonstrating why finite automata alone are insufficient for recognizing every programming-language structure.

Question 5. Explain the Closure Properties of Regular Languages and Their Use in Combining Detection Rules

Answer:

Closure properties describe how a language class behaves when operations are performed on its languages. Regular languages are closed under several operations, meaning that applying these operations to regular languages produces another regular language.

These properties are useful in intrusion detection, spam filtering, and input validation, where multiple pattern-matching rules may need to be combined.

1. Important closure properties

Let (L1) and (L2) be regular languages.

Operation	Meaning	Example application
Union (L1 cup L2)	Accept strings belonging to either language	Detecting any one of several suspicious patterns
Intersection (L1 cap L2)	Accept strings belonging to both languages	Checking whether two pattern conditions are satisfied
Complement (overline{L1})	Accept strings not belonging to a language	Rejecting inputs outside an allowed pattern
Difference (L1-L2)	Accept strings in (L1) but not (L2)	Excluding known safe patterns from a broader match
Concatenation (L1L2)	Join a string from (L1) with one from (L2)	Recognizing sequences of patterns
Kleene star (L1^*)	Zero or more repetitions of strings	Recognizing repeated commands or

	from (L1)	symbols

2. How multiple detection rules can be combined

Consider an intrusion detection system with two regular languages:

- (L1): Strings containing a pattern associated with a suspicious command.
- (L2): Strings containing a pattern associated with a suspicious URL.

To detect either pattern, the system can use the union:

[L=L1 cup L2]

A finite automaton can be constructed to recognize the union of the two languages. An input matching either rule will be accepted.

If both conditions must be satisfied, the intersection can be used:

[L=L1 cap L2]

This allows the system to combine individual rules into a larger pattern-recognition mechanism.

3. Why the resulting language remains regular

The resulting language remains regular because regular languages are closed under these operations.

- For union, an NFA can be constructed with transitions that allow either language to be recognized.
- For intersection, a product DFA can track the current states of both automata simultaneously.
- For complement, the accepting and non-accepting states of a complete DFA can be exchanged.
- For concatenation and Kleene star, an equivalent NFA can be constructed using epsilon transitions where necessary.

Since these constructions produce finite automata, the resulting languages remain regular.

4. Advantages in real-world systems

Modularity: Each detection rule can be designed and tested separately.

Efficiency: Finite automata can process input strings through state transitions, often in time proportional to the input length for a fixed automaton.

Scalability: New regular-pattern rules can be incorporated into a detection system without changing the fundamental recognition model.

Consistency: Closure properties provide a mathematical basis for combining rules while preserving regular-language recognition.

However, regular expressions and finite automata do not detect every possible security threat. Complex threats may require additional parsing, contextual analysis, or other detection techniques.

Conclusion: Closure properties allow multiple regular-language detection rules to be combined while preserving regularity. This makes finite automata useful for building modular and efficient pattern-matching systems.